CONTRACT WOLF

Security Assessment

DARKPOOL

Verified on October 16, 2026

SUMMARY

Project	CHAIN	METHODOLOGY
DARKPOOL	Robinhood Chain	Manual & Automatic Analysis

FILES	DELIVERY	TYPE
Repository	October 16, 2026	Standard Audit



12	0	2	2	6	2	12
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

🟥 0 Critical	An exposure that can affect the contract functions in several events that can risk and disrupt the contract
🟧 2 Major	An opening & exposure to manipulate the contract in an unwanted manner
🟨 2 Medium	An opening that could affect the outcome in executing the contract in a specific situation
⬛ 6 Minor	An opening but doesn't have an impact on the functionality of the contract
🟦 2 Informational	An opening that consists information but will not risk or affect the contract
🟩 12 Resolved	ContractWolf's findings has been acknowledged & resolved by the project

STATUS ✔️**AUDIT PASSED**

TABLE OF CONTENTS | DARKPOOL

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details
Inheritance Graph
Call Graph

Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

DISCLAIMER | DARKPOOL

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | DARKPOOL

DARKPOOL team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **DARKPOOL**.

AUDITING APPROACH | DARKPOOL

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
   - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
   - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.

2. Testing and automated analysis that includes :
   - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.

3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.

4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | DARKPOOL



DARKPOOL is a shielded pool for private trading. Users shield coins into private notes, swap privately through Jupiter (Solana) or Uniswap V3/V4 (Robinhood Chain), send privately and unshield to any address.

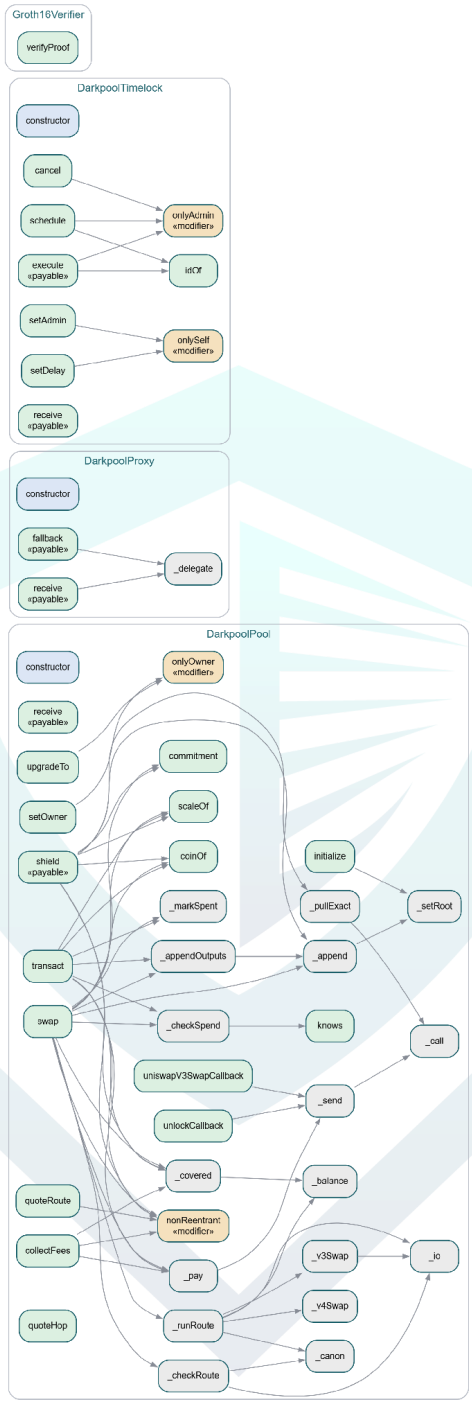
Token Name	Symbol	Decimal	Total Supply	Chain
DARKPOOL	DARKPOOL	6	-	Robinhood Chain

SOURCE

Source	*GitHub Repository (darkpoolsol/darkpool-audit, commit 71b1a76) and Pool Proxy 0xEB66bd2d7d29f1F29a19C88cE4b2035D4571076E*

CALL GRAPH | DARKPOOL

Call Graph of Contract Functions



FINDINGS | DARKPOOL



12	0	2	2	6	2	12
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

This report has been prepared to state the issues and vulnerabilities for DARKPOOL through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
CEREMONY-GATE	The Robinhood deploy trusts a flag instead of hashing the ceremony artifacts	scripts/rh-deploy.mjs, L: 30-35	Major	Resolved
SETUP-TRUST	Phase 2 of the ceremony has a single contributor	ceremony/transcript.json, L: 11-32	Major	Resolved
ROUTE-UNVERIFIED	Any contract can act as a V3 route pool, and V4 keys are unchecked	evm/src/DarkpoolPool.sol, L: 301-311	Medium	Resolved
UPGRADE-NOCODE	upgradeTo accepts an address that holds no code	evm/src/DarkpoolPool.sol, L: 138-141	Medium	Resolved
WETH-STRANDED	A V3 hop refunds unspent WETH from the pool's own report	evm/src/DarkpoolPool.sol, L: 335-341	Minor	Resolved
CW-003	No path to recover ETH or tokens that back no note	evm/src/DarkpoolPool.sol, L: 135, 263-271	Minor	Resolved
SWC-113	Either fee recipient can block every future fee collection	evm/src/DarkpoolPool.sol, L: 263-271	Minor	Resolved
TIMELOCK-NOGRACE	A scheduled timelock call never expires	evm/src/DarkpoolTimelock.sol, L: 45-55	Minor	Resolved

TOKEN-GATE	Any ERC-20 can be shielded, with no listing check	evm/src/DarkpoolPool.sol, L: 150-174	Minor	Resolved
DEPLOY-OWNER	The deploy leaves instant upgrade power with a hot key	scripts/rh-deploy.mjs, L: 57-62	Minor	Resolved
PROXY-INIT	DarkpoolProxy does not require a non-empty initializer	evm/src/DarkpoolPool.sol, L: 461-465	Informational	Resolved
QUOTE-ONCHAIN	quoteRoute is an unrestricted state-touching entry point	evm/src/DarkpoolPool.sol, L: 274-290	Informational	Resolved

SWC ATTACKS | DARKPOOL

Smart Contract Weakness Classification and Test Cases

ID	Description	Status
SWC-100	Function Default Visibility	Passed
SWC-101	Integer Overflow and Underflow	Passed
SWC-102	Outdated Compiler Version	Passed
SWC-103	Floating Pragma	Passed
SWC-104	Unchecked Call Return Value	Passed
SWC-105	Unprotected Ether Withdrawal	Passed
SWC-106	Unprotected SELF DESTRUCT Instruction	Passed
SWC-107	Reentrancy	Passed
SWC-108	State Variable Default Visibility	Passed
SWC-109	Uninitialized Storage Pointer	Passed
SWC-110	Assert Violation	Passed
SWC-111	Use of Deprecated Solidity Functions	Passed
SWC-112	Delegate call to Untrusted Callee	Passed
SWC-113	DoS with Failed Call	Not Passed
SWC-114	Transaction Order Dependence	Passed
SWC-115	Authorization through tx.origin	Passed
SWC-116	Block values as a proxy for time	Passed
SWC-117	Signature Malleability	Passed
SWC-118	Incorrect Constructor Name	Passed
SWC-119	Shadowing State Variables	Passed
SWC-120	Weak Sources of Randomness from Chain Attributes	Passed
SWC-121	Missing Protection against Signature Replay Attacks	Passed
SWC-122	Lack of Proper Signature Verification	Passed

ID	Description	Status
SWC-123	Requirement Violation	Passed
SWC-124	Write to Arbitrary Storage Location	Passed
SWC-125	Incorrect Inheritance Order	Passed
SWC-126	Insufficient Gas Griefing	Passed
SWC-127	Arbitrary Jump with Function Type Variable	Passed
SWC-128	DoS With Block Gas Limit	Passed
SWC-129	Typographical Error	Passed
SWC-130	Right-To-Left-Override control character(U+202E)	Passed
SWC-131	Presence of unused variables	Passed
SWC-132	Unexpected Ether balance	Passed
SWC-133	Hash Collisions With Multiple Variable Arguments	Passed
SWC-134	Message call with hardcoded gas amount	Passed
SWC-135	Code With No Effects	Passed
SWC-136	Unencrypted Private Data On-Chain	Passed

CW ASSESSMENT | DARKPOOL

ContractWolf Vulnerability and Security Tests

ID	Name	Description	Status
CW-001	Multiple Version	Presence of multiple compiler version across all contracts	✔
CW-002	Incorrect Access Control	Additional checks for critical logic and flow	✘
CW-003	Payable Contract	A function to withdraw ether should exist otherwise the ether will be trapped	✘
CW-004	Custom Modifier	major recheck for custom modifier logic	✔
CW-005	Divide Before Multiply	Performing multiplication before division is generally better to avoid loss of precision	✘
CW-006	Multiple Calls	Functions with multiple internal calls	✘
CW-007	Deprecated Keywords	Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions	✔
CW-008	Unused Contract	Presence of an unused, unimported or uncalled contract	✔
CW-009	Assembly Usage	Use of EVM assembly is error-prone and should be avoided or double-checked for correctness	✘
CW-010	Similar Variable Names	Variables with similar names could be confused for each other and therefore should be avoided	✘
CW-011	Commented Code	Removal of commented/unused code lines	✔
CW-012	SafeMath Override	SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking.	✔

FIXES & RECOMMENDATION

CEREMONY-GATE | The Robinhood deploy trusts a flag instead of hashing the ceremony artifacts

The only provenance check before the verifier is generated and deployed is `setup.ceremony !== true`, a boolean in a local JSON file. The Solana script re-hashes every artifact against `transcript.json` and refuses to broadcast on a mismatch, but this one does not. Any directory holding a `transact.zkey` and a one-line `setup.json` deploys as the ceremony's verifier, including the test setup whose secret still exists on a developer machine.

```
const setup = JSON.parse(await readFile(join(zkDir, 'setup.json'), 'utf8'));
if (setup.ceremony !== true) throw Error(`${zkDir} is not a finalized ceremony's output`);
const build = resolve('.local/rh-deploy/evm');
// ...
execFileSync(process.execPath, ['evm/scripts/verifier.mjs', zkDir,
                                join(build, 'src/Verifier.sol')], {stdio: 'inherit'});
```

Recommendation

Hash `transact.zkey`, `transact.wasm` and `transact-vk.json` and require each to equal `transcript.json`'s final record before the verifier is built, exactly as `scripts/dp-deploy.mjs` does, and refuse to broadcast otherwise.

```
const t = JSON.parse(await readFile(resolve(zkDir, '..', 'transcript.json'), 'utf8'));
const finalized = setup.ceremony === true && t?.final?.artifacts
  && JSON.stringify(t.final.artifacts) === JSON.stringify(setup.artifacts);
for (const [f, want] of Object.entries(t?.final?.artifacts ?? {})) {
  const got = createHash('sha256').update(await readFile(join(zkDir, f))).digest('hex');
  if (got !== want) throw Error(`${f} does not match the ceremony transcript (${got})`);
}
if (!finalized) throw Error(`${zkDir} is not a finalized DARKPOOL ceremony. Refusing to
deploy.`);
```

SETUP-TRUST | Phase 2 of the ceremony has a single contributor

One circuit authorizes every spend on both chains, and its transcript records one phase-2 contribution, by the DARKPOOL builder's launch machine, followed by a Bitcoin block beacon, all inside one 32 second run on one machine. A beacon's exponent comes from public data, so it adds no secrecy: whoever kept that contribution's secret can reconstruct the final `delta` and forge proofs against this pool.

```
"steps": [
  { "step": 0, "kind": "init",         "at": "2026-10-03T04:32:00.908Z" },
  { "step": 1, "kind": "contribution", "at": "2026-10-03T04:32:02.194Z",
    "name": "DARKPOOL builder (launch machine)" },
  { "step": 2, "kind": "beacon",       "at": "2026-10-03T04:32:11.828Z",
    "source": "Bitcoin block 969674" }
]
```

Recommendation

Re-run phase 2 with several independent contributors who share neither a machine nor an operator, publish each contribution hash and attestation as it is made, then re-finalize and deploy a fresh verifier behind the timelock.

```
# phase 2, one pass per independent contributor, each on their own machine
snarkjs zkey contribute transact_0000.zkey transact_0001.zkey \
  -n="contributor 1" -e="<entropy only contributor 1 sees>"
snarkjs zkey contribute transact_0001.zkey transact_0002.zkey \
  -n="contributor 2 (independent)" -e="<entropy only contributor 2 sees>"
snarkjs zkey verify transact.r1cs pot16_final.ptau transact_0002.zkey
# the beacon is the last step, never the only source of randomness
snarkjs zkey beacon transact_0002.zkey transact.zkey <block-hash> 10
```

ROUTE-UNVERIFIED | Any contract can act as a V3 route pool, and V4 keys are unchecked

`_checkRoute` only requires `hops[i].pool.code.length != 0` for a V3 hop and reads `token0()` and `token1()` from it; for a V4 hop it reads the caller's own `PoolKey` and never looks at `hooks`. The pool then calls that address, and while it holds control `uniswapV3SwapCallback` is open to it. The balance deltas in `_runRoute`, the `nonReentrant` guard and the absence of any ERC-20 approval keep the loss to the caller's own `tradeIn`, so every guarantee rests on those deltas alone.

```
function _checkRoute(SwapArgs calldata w, Hop[] calldata hops) private view {
    if (hops.length == 0 || hops.length > 3) revert BadRoute();
    address cur = w.tokenIn;
    for (uint256 i = 0; i < hops.length; i++) {
        if (hops[i].v3 && hops[i].pool.code.length == 0) revert BadRoute();
        (address cIn, address cOut) = _io(hops[i]);
        if (_canon(cIn) != cur || (i > 0 && cur == ETH)) revert BadRoute();
        cur = _canon(cOut);
    }
    if (cur != w.tokenOut) revert BadRoute();
}
```

Recommendation

Confirm each V3 hop against the Uniswap V3 factory's `getPool` for its own tokens and fee, and require a V4 key's `hooks` to be the zero address or one of an owner-approved set, so the pool never calls an arbitrary contract.

```
IUniswapV3Factory constant V3_FACTORY =
IUniswapV3Factory(0x1F98431c8aD98523631AE4a59f267346ea31F984);

if (hops[i].v3) {
    IUniswapV3Pool p = IUniswapV3Pool(hops[i].pool);
    if (V3_FACTORY.getPool(p.token0(), p.token1(), p.fee()) != hops[i].pool) revert
BadRoute();
} else if (hops[i].key.hooks != address(0) && !approvedHook[hops[i].key.hooks]) {
    revert BadRoute();
}
```

UPGRADE-NOCODE | upgradeTo accepts an address that holds no code

`upgradeTo` writes the implementation slot with a raw `sstore` and never checks that the address holds code. A `delegatecall` to an account with no code succeeds and returns nothing, so the proxy would answer every call silently, `owner` would read as zero, and no further `upgradeTo` could ever be made. Every shielded balance in the pool would be unreachable, with no recovery path at all.

```
function upgradeTo(address implementation) external onlyOwner {
    assembly { sstore(IMPL_SLOT, implementation) }
    emit Upgraded(implementation);
}
```

Recommendation

Require the new implementation to hold code before writing the slot. One line turns an unrecoverable mistake into a reverted transaction, and the timelock already gives the owner time to notice.

```
function upgradeTo(address implementation) external onlyOwner {
    if (implementation.code.length == 0) revert Unauthorized();
    assembly { sstore(IMPL_SLOT, implementation) }
    emit Upgraded(implementation);
}
```

WETH-STRANDED | A V3 hop refunds unspent WETH from the pool's own report

`_v3Swap` unwraps `amountIn - paid` using the `paid` the route pool returned, while the amount actually handed over was `owed` inside `uniswapV3SwapCallback`. A pool that reports a `paid` larger than the `owed` it took leaves the difference wrapped as WETH in the contract. `shield` rejects WETH and `shielded[weth]` is always zero, so nothing in the pool can reach it again.

```
(int256 a0, int256 a1) = IUniswapV3Pool(h.pool).swap(address(this), h.zeroForOne,
    int256(amountIn), h.zeroForOne ? MIN_SQRT : MAX_SQRT, "");
v3Pool = address(0); v3Max = 0;
(int256 paid, int256 o) = h.zeroForOne ? (a0, a1) : (a1, a0);
if (o > 0 || paid < 0 || uint256(paid) > amountIn) revert BadRoute();
out = uint256(-o);
if (cIn == weth && uint256(paid) < amountIn) IWETH(weth).withdraw(amountIn -
uint256(paid));
```

Recommendation

Record what the callback actually paid and unwrap against that, so the refund follows the real transfer rather than the pool's report of it.

```
uint256 private v3Paid;
// in uniswapV3SwapCallback, just before _send:
//     v3Paid = owed;

v3Paid = 0;
(int256 a0, int256 a1) = IUniswapV3Pool(h.pool).swap(/* ... */);
uint256 took = v3Paid;
v3Pool = address(0); v3Max = 0; v3Paid = 0;
// ...
if (cIn == weth && took < amountIn) IWETH(weth).withdraw(amountIn - took);
```

CW-003 | No path to recover ETH or tokens that back no note

The pool is payable through `receive()`, and ETH sent straight to the proxy, WETH stranded by a route, and any token transferred in directly all raise `_balance` without ever raising `shielded`. `_covered` only checks a lower bound, so the surplus sits there permanently, and `collectFees` pays out `ethFees` alone. Only a timelocked upgrade can ever free it.

```
receive() external payable {} // ETH from the PoolManager and WETH

function _covered(address token) private view {
    if (_balance(token) < shielded[token] + (token == ETH ? ethFees : 0))
        revert Undercollateralized();
}
```

Recommendation

Add an owner-only sweep that can move only the surplus above what the coin's notes and fees are owed, so stuck funds are recoverable while shielded balances stay untouchable.

```
/// Only what no note and no fee is owed: shielded balances can never be swept.
function sweepSurplus(address token, address to) external onlyOwner nonReentrant {
    uint256 owed = shielded[token] + (token == ETH ? ethFees : 0);
    uint256 have = _balance(token);
    if (have <= owed) revert BadAmount();
    _pay(token, to, have - owed);
    _covered(token);
}
```

SWC-113 | Either fee recipient can block every future fee collection

`collectFees` pays `builder` and then `feeWallet` with `_pay`, which reverts the whole call if either transfer fails. `feeWallet` and `builder` are fixed in `initialize` and the contract exposes no setter for them. If either address becomes a contract that rejects ETH or exhausts the forwarded gas, every later collection reverts and `ethFees` can only be released by a timelocked upgrade.

```
function collectFees() external nonReentrant {
    uint256 amount = ethFees; uint256 toBuilder = builderFees;
    if (amount == 0 || toBuilder > amount) revert BadAmount();
    ethFees = 0; builderFees = 0;
    _pay(ETH, builder, toBuilder);
    _pay(ETH, feeWallet, amount - toBuilder);
    emit FeesCollected(amount, toBuilder);
    _covered(ETH);
}
```

Recommendation

Credit each recipient to a claim balance and let them withdraw, so one unreachable address cannot hold up the other's share or the accounting.

```
mapping(address => uint256) public claimable;

function collectFees() external nonReentrant {
    uint256 amount = ethFees; uint256 toBuilder = builderFees;
    if (amount == 0 || toBuilder > amount) revert BadAmount();
    ethFees = 0; builderFees = 0;
    claimable[builder] += toBuilder;
    claimable[feeWallet] += amount - toBuilder;
    emit FeesCollected(amount, toBuilder);
}
function claim() external nonReentrant {
    uint256 owed = claimable[msg.sender];
    claimable[msg.sender] = 0;
    _pay(ETH, msg.sender, owed);
    _covered(ETH);
}
```

TIMELOCK-NOGRACE | A scheduled timelock call never expires

`execute` only checks that `eta[id]` is set and has passed, with no upper bound. The 3 day delay gives the public notice when a call is scheduled, but the call stays runnable forever, so a proposal made months earlier can be executed at a moment nobody is watching. That weakens the stated promise that anyone can see a change coming and leave first.

```
function execute(address target, uint256 value, bytes calldata data, bytes32 salt)
    external payable onlyAdmin returns (bytes memory) {
    bytes32 id = idOf(target, value, data, salt);
    uint256 t = eta[id];
    if (t == 0) revert NotScheduled();
    if (block.timestamp < t) revert TooEarly();
    delete eta[id];
    (bool ok, bytes memory r) = target.call{value: value}(data);
    if (!ok) revert CallFailed(r);
    // ...
}
```

Recommendation

Add a grace period after which a scheduled call lapses and has to be scheduled again, which puts every execution inside a bounded, recently announced window.

```
uint256 public constant GRACE_PERIOD = 14 days;
error Stale();

if (t == 0) revert NotScheduled();
if (block.timestamp < t) revert TooEarly();
if (block.timestamp > t + GRACE_PERIOD) { delete eta[id]; revert Stale(); }
```

TOKEN-GATE | Any ERC-20 can be shielded, with no listing check

The Solana program gates deposits through `listable`, but `shield` accepts any address but WETH. `_pullExact` turns away fee-on-transfer tokens and per-coin `shielded` keeps the trouble to one coin, yet a token that can freeze, blacklist, seize or rebase the pool's balance down makes `_covered(token)` fail and leaves every note of it unspendable. A token with a mutable `decimals()` does the same through `scaleOf`.

```
function shield(address token, uint256 units, uint256 hide, bytes calldata record)
    external payable nonReentrant {
    if (units == 0 || units > type(uint64).max || hide >= FIELD) revert BadAmount();
    if (record.length != RECORD) revert BadRecord();
    if (token == weth) revert BadRoute(); // shield ETH instead
    uint256 amount = units * scaleOf(token);
    // ...
}
```

Recommendation

Keep an owner-approved list of shieldable tokens, as the Solana side does with `listable`, and pin each token's scale on first listing so a later `decimals()` change cannot move the accounting.

```
mapping(address => uint256) public listedScale; // 0: not listed

function listToken(address token) external onlyOwner {
    if (token == weth || listedScale[token] != 0) revert BadRoute();
    listedScale[token] = scaleOf(token);          // pinned once, never re-read
}
// in shield and everywhere scaleOf was used for an ERC-20:
uint256 k = token == ETH ? 1e9 : listedScale[token];
if (k == 0) revert BadRoute();
```

DEPLOY-OWNER | The deploy leaves instant upgrade power with a hot key

The proxy is initialized with the deployer's own address as `owner`, and the script never calls `setOwner` to hand that over to `DarkpoolTimelock`. Its read-back check even asserts the deployer is the owner, so a passing run is one that ends with a single hot key able to `upgradeTo` arbitrary code with no delay. The handover is left to a separate manual step.

```
const proxy = await deploy('Pool (proxy)', X.abi, X.bytecode.object, impl,
    POOL_IFACE.encodeFunctionData('initialize', [deployerKey.address, keeperAddr,
builder]));
// ...
if ((await dp.owner()) !== deployerKey.address || /* ... */)
    throw Error('the pool did not initialize as intended');
```

Recommendation

Take a `--timelock` address, hand ownership over in the same run, and make the read-back assert the timelock owns the pool, so no run can finish with the deploy key still in charge.

```
const timelock = arg('timelock');
if (!ethers.isAddress(timelock ?? '')) throw Error('pass --timelock <0x...>');
// right after the proxy is deployed and read back:
await (await poolAt(proxy, deployer).setOwner(timelock)).wait();
if ((await dp.owner()) !== ethers.getAddress(timelock))
    throw Error('ownership was not handed to the timelock');
```

PROXY-INIT | DarkpoolProxy does not require a non-empty initializer

The constructor `delegatecall`s `init` and reverts only when that call fails. With empty calldata it reaches the implementation's `receive()`, which succeeds, so the proxy would deploy uninitialized and `initialize` would be open to anyone, and with it `upgradeTo`. `scripts/rh-deploy.mjs` always passes the encoded `initialize`, so the live proxy is not affected.

```
constructor(address implementation, bytes memory init) {
    assembly { sstore(IMPL_SLOT, implementation) }
    (bool ok, bytes memory r) = implementation.delegatecall(init);
    if (!ok) assembly { revert(add(r, 32), mload(r)) }
}
```

Recommendation

Require a non-empty initializer and confirm the slot was written, so a proxy can never exist in a state where anyone may claim it.

```
constructor(address implementation, bytes memory init) {
    require(implementation.code.length != 0 && init.length >= 4, "bad init");
    assembly { sstore(IMPL_SLOT, implementation) }
    (bool ok, bytes memory r) = implementation.delegatecall(init);
    if (!ok) assembly { revert(add(r, 32), mload(r)) }
    require(DarkpoolPool(payable(address(this))).initialized(), "not initialized");
}
```

QUOTE-ONCHAIN | quoteRoute is an unrestricted state-touching entry point

`quoteRoute` is meant for `eth_call`, but it is not `view` and anyone can send it as a transaction. `quoteHop` writes `v3Pool` and `v3Quote` and relies on its own closing `revert` to roll those writes back. It works, yet it lets any caller drive arbitrary external `swap()` calls from the pool's own address, and the pool's callback state now depends on a revert always happening.

```
function quoteRoute(Hop[] calldata hops, uint256 amountIn) external nonReentrant
    returns (uint256 amount) {
    amount = amountIn;
    for (uint256 i = 0; i < hops.length; i++) {
        try this.quoteHop(hops[i], amount) { revert BadRoute(); }
        catch (bytes memory r) { /* reads QuoteResult */ }
    }
}
```

Recommendation

Clear `v3Pool` and `v3Quote` explicitly in `quoteHop` before it reverts, and move quoting to a separate stateless quoter contract so the pool itself is never driven by an unauthenticated caller.

```
function quoteHop(Hop calldata h, uint256 amountIn) external {
    if (msg.sender != address(this)) revert Unauthorized();
    if (h.v3) {
        v3Pool = h.pool; v3Quote = true;
        IUniswapV3Pool(h.pool).swap(/* ... */);
    } else poolManager.unlock(abi.encode(true, h.key, h.zeroForOne, amountIn));
    v3Pool = address(0); v3Quote = false;   // never left set, revert or not
    revert BadRoute();
}
```

AUDIT COMMENTS | DARKPOOL

Smart Contract audit comment for a non-technical perspective

- The pool sits behind an ERC-1967 proxy and is upgradeable. Its owner is documented as `DarkpoolTimelock` with a 3 day delay, whose admin is a 2-of-3 Safe, so an upgrade can replace the whole contract after 3 days of public notice.
- `upgradeTo` accepts an address with no code. One such upgrade would leave the proxy answering every call silently, with every shielded balance unreachable and no way to upgrade again.
- Proof soundness rests on one phase-2 ceremony contribution, made by the project on its own machine. Whoever holds that secret could forge a spend against this pool.
- The Robinhood deploy script checks only a boolean in a local file before building the on-chain verifier, so nothing in the code stops a non-ceremony proving key being deployed.
- Any ERC-20 can be shielded. A token that can freeze, blacklist or seize the pool's holding of it will lock every note of that coin, though never notes of any other coin.
- ETH or tokens sent to the pool that back no note cannot be recovered, because the contract has no sweep and `collectFees` pays out only the accrued swap fees.
- The owner cannot touch a holder's notes, move funds, pause the pool or change the fee. Its only powers are `upgradeTo` and `setOwner`, and both go through the timelock.
- The 0.5% swap fee and the 80/20 split between the fee wallet and the builder are `constant`, and `feeWallet` and `builder` are fixed at `initialize` with no setter.
- Every spend binds the chain id, this contract's address, the relayer, the fee, the expiry, the recipient or the whole route and the records into the proof, so a proof cannot be replayed on another chain or reused by another relayer.
- Anyone may call `collectFees`. It pays only the two fixed addresses and takes nothing for the caller.
- Swap amounts are measured from the contract's real balance changes rather than from what a route reports, the pool grants no ERC-20 approval to anyone, and each coin must still cover its own notes after every action.
- The pool runs every swap hop itself, so no outside router ever holds the funds in transit.



CONTRACTWOLF

Blockchain Security - Smart Contract Audits